

JobVago Test Case Documentation

Complete Test Suite — 278 Test Cases

Item	Details
Project	JobVago — Career Guidance Web Application
Total Test Cases	278 (all passing)
Test Files	tests/controllers/test_auth_controller.py (9) tests/controllers/test_dashboard_controller.py (9) tests/models/test_career.py (10) tests/models/test_user.py (22) tests/services/test_auth_service.py (15) tests/services/test_course_service.py (34) tests/services/test_data_downloader.py (9) tests/services/test_data_service.py (13) tests/services/test_favorite_course_service.py (15) tests/services/test_industry_keywords.py (11) tests/services/test_industry_service.py (9) tests/services/test_resume_analysis_service.py (8) tests/services/test_resume_grading_service.py (28) tests/services/test_skill_extraction_service.py (12) tests/test_basis_path.py (15) tests/test_ec_bva_auth_controller.py (37)
Testing Techniques	1. White Box — Basis Path Testing 2. Black Box — Equivalence Class Partitioning 3. Black Box — Boundary Value Analysis 4. Black Box — Functional Testing

Table of Contents

1. White Box Testing — Basis Path Testing (15 tests)
 - 1.1 RuleBasedScorer.score_specifics() — 6 tests
 - 1.2 CourseService._apply_filters() — 9 tests
2. Black Box Testing — Equivalence Class Partitioning (26 tests)
 - 2.1 POST /auth/register — 13 tests
 - 2.2 POST /auth/login — 7 tests
 - 2.3 Auth Guard Routes — 6 tests
3. Black Box Testing — Boundary Value Analysis (11 tests)
 - 3.1 Registration Password Boundary — 3 tests
 - 3.2 Registration Name Boundary — 5 tests
 - 3.3 Login Password Boundaries — 3 tests
4. Black Box — Functional Testing: Controllers (18 tests)
 - 4.1 Auth Controller Routes — 9 tests
 - 4.2 Dashboard & Public Routes — 9 tests
5. Black Box — Functional Testing: Models (32 tests)
 - 5.1 User Model — 8 tests
 - 5.2 Favorite Courses (via model) — 6 tests
 - 5.3 User Skills — 3 tests

5.4 Resume Analysis Model — 5 tests

5.5 Career Models (Industry, Course, DashboardStats) — 10 tests

6. Black Box — Functional Testing: Services (176 tests)

6.1 AuthenticationService — 15 tests

6.2 CourseService — 34 tests

6.3 DataDownloader — 9 tests

6.4 DataService — 13 tests

6.5 FavoriteCourseService — 15 tests

6.6 IndustryKeywords — 11 tests

6.7 IndustryService — 9 tests

6.8 ResumeAnalysisService — 8 tests

6.9 ResumeGradingService — 28 tests

6.10 SkillExtractionService — 12 tests

6.11 Resume Grading (Full Pipeline) — 3 tests

6.12 AI Scorer Fallbacks — 7 tests

1. White Box Testing — Basis Path Testing

Basis path testing (McCabe's technique) derives test cases from the control flow graph (CFG). The cyclomatic complexity $V(G)$ determines the number of linearly independent paths. One test case is written per path to guarantee full edge coverage.

Source file: tests/test_basis_path.py

1.1 RuleBasedScorer.score_specifics(text)

Location: app/services/resume_grading_service.py | $V(G) = 6$ | 6 independent paths

Test Input	Expected Output	Actual Output
Path 1: text="" (empty)	points=0, quantified=0, total=0	points=0, quantified=0, total=0 PASSED
Path 2: 5 bullets, 4 with numbers ratio=4/5=0.80 ≥ 0.6	points=8, quantified=4, total=5	points=8, quantified=4, total=5 PASSED
Path 3: 5 bullets, 2 with numbers ratio=2/5=0.40 (boundary)	points=6, quantified=2, total=5	points=6, quantified=2, total=5 PASSED
Path 4: 5 bullets, 1 with number ratio=1/5=0.20 (boundary)	points=4, quantified=1, total=5	points=4, quantified=1, total=5 PASSED
Path 5: 6 bullets, 1 with number ratio=1/6=0.167, quantified>0	points=2, quantified=1, total=6	points=2, quantified=1, total=6 PASSED
Path 6: 3 bullets, 0 with numbers quantified=0	points=0, quantified=0, total=3	points=0, quantified=0, total=3 PASSED

1.2 CourseService._apply_filters(courses, filters)

Location: app/services/course_service.py | $V(G) = 9$ | 9 independent paths

Test Input	Expected Output	Actual Output
Path 1: filters=None	Return original list, len=4	len=4, same object PASSED
Path 2: {sort_by: 'Highest Rating'}	len=4, sorted desc by rating: 4.5, 4.0, 3.5, 3.0	[4.5, 4.0, 3.5, 3.0] PASSED
Path 3: {impact_level: 'High Impact'}	len=2, only High Impact	len=2, all High Impact PASSED
Path 4: {impact_level: 'All Impact Levels'}	len=4, filter skipped	len=4 PASSED
Path 5: {mode: 'Full-time'}	len=2, only Full-time	len=2, all Full-time PASSED
Path 6: {mode: 'All Modes'}	len=4, filter skipped	len=4 PASSED
Path 7: {language: 'Mandarin'}	len=1, Java Masterclass only	len=1, title=Java Masterclass PASSED
Path 8: {sort_by: 'Lowest Price'}	len=4, sorted asc by price	Prices: [60, 80, 100, 200] PASSED
Path 9: {sort_by: 'Shortest Duration'}	len=4, sorted asc by hours	10h, 15h, 20h, 30h PASSED

2. Black Box Testing — Equivalence Class Partitioning

Source file: tests/test_ec_bva_auth_controller.py

Target: app/controllers/auth_controller.py

2.1 POST /auth/register — Equivalence Classes (13 tests)

Test Input	Expected Output	Actual Output
EC1: email='alice@example.com' (valid)	HTTP 302 redirect to dashboard	HTTP 302 PASSED
EC2: email='invalidemail.com' (no @)	HTTP 200, error msg	HTTP 200 PASSED
EC3: email='' (empty)	HTTP 200, rejected	HTTP 200 PASSED
EC4: email='test@example.com' (duplicate)	HTTP 200, 'already registered'	HTTP 200, 'already registered' PASSED
EC5: email=' ALICE@EXAMPLE.COM '	HTTP 302, normalised to lowercase	HTTP 302, login works PASSED
EC6: password='abcdefgh' (len=8, valid)	HTTP 302	HTTP 302 PASSED
EC7: password='abc' (len=3, too short)	HTTP 200, '6 characters'	HTTP 200, '6 characters' PASSED
EC8: password='' (empty)	HTTP 200, rejected	HTTP 200 PASSED
EC9: confirm_password matches password	HTTP 302	HTTP 302 PASSED
EC10: confirm_password differs	HTTP 200, 'do not match'	HTTP 200, 'do not match' PASSED
EC11: full_name='Alice Smith' (valid)	HTTP 302	HTTP 302 PASSED
EC12: full_name='A' (too short)	HTTP 200, 'name' error	HTTP 200, 'name' PASSED
EC13: full_name='' (empty)	HTTP 200, rejected	HTTP 200 PASSED

2.2 POST /auth/login — Equivalence Classes (7 tests)

Test Input	Expected Output	Actual Output
EC14: Valid credentials	HTTP 302 redirect	HTTP 302 PASSED
EC15: Unregistered email	HTTP 200, 'invalid'	HTTP 200, 'invalid' PASSED
EC16: Empty email	HTTP 200, rejected	HTTP 200 PASSED
EC17: email='TEST@EXAMPLE.COM'	HTTP 302, case-insensitive	HTTP 302 PASSED
EC18: Correct password	HTTP 200 (dashboard after redirect)	HTTP 200 PASSED
EC19: Wrong password	HTTP 200, 'invalid'	HTTP 200, 'invalid' PASSED
EC20: Empty password	HTTP 200, rejected	HTTP 200 PASSED

2.3 Auth Guard Routes — Equivalence Classes (6 tests)

Test Input	Expected Output	Actual Output
EC21: Authenticated user GET /auth/login	HTTP 302 to dashboard	HTTP 302 PASSED
EC22: Authenticated user GET /auth/register	HTTP 302 to dashboard	HTTP 302 PASSED
EC23: Unauthenticated user GET /auth/login	HTTP 200, form rendered	HTTP 200 PASSED
EC24: Unauthenticated user GET /auth/register	HTTP 200, form rendered	HTTP 200 PASSED
EC25: Authenticated user GET /auth/logout	HTTP 302 to public page	HTTP 302 PASSED
EC26: Login with ?next=/resume/upload	HTTP 302 to /resume/upload	HTTP 302, Location=/resume/upload PASSED

3. Black Box Testing — Boundary Value Analysis

Source file: tests/test_ec_bva_auth_controller.py

Target: auth_controller + auth_service

3.1 Registration Password Boundary (min 6 chars) — 3 tests

Test Input	Expected Output	Actual Output
BV1: password='abcde' (len=5, below)	HTTP 200, '6 characters'	HTTP 200, '6 characters' PASSED
BV2: password='abcdef' (len=6, on)	HTTP 302 (accepted)	HTTP 302 PASSED
BV3: password='abcdefg' (len=7, above)	HTTP 302 (accepted)	HTTP 302 PASSED

3.2 Registration Name Boundary (min 2 chars stripped) — 5 tests

Test Input	Expected Output	Actual Output
BV4: full_name='X' (stripped len=1)	HTTP 200, 'name' error	HTTP 200, 'name' PASSED
BV5: full_name='Jo' (stripped len=2)	HTTP 302 (accepted)	HTTP 302 PASSED
BV6: full_name='Joe' (stripped len=3)	HTTP 302 (accepted)	HTTP 302 PASSED
BV7: full_name='' (stripped len=0)	HTTP 200, rejected	HTTP 200 PASSED
BV8: full_name=' A ' (stripped len=1)	HTTP 200, 'name' error	HTTP 200, 'name' PASSED

3.3 Login Password Boundaries — 3 tests

Test Input	Expected Output	Actual Output
BV9: Register with 'abcdef' (len=6), login same	HTTP 302 (hash verified at boundary)	HTTP 302 PASSED
BV10: Correct='password123', try 'password12'	HTTP 200, 'invalid' (1 char short)	HTTP 200, 'invalid' PASSED
BV11: Correct='password123', try 'password1234'	HTTP 200, 'invalid' (1 char extra)	HTTP 200, 'invalid' PASSED

4. Black Box — Functional Testing: Controllers

These test cases use functional / black-box testing to verify controller routes return the correct HTTP status codes and response content.

Source files: tests/controllers/test_auth_controller.py, tests/controllers/test_dashboard_controller.py

4.1 Auth Controller Routes (9 tests)

File: tests/controllers/test_auth_controller.py

Test Input	Expected Output	Actual Output
GET /auth/login	HTTP 200, page contains login text	HTTP 200 PASSED
GET /auth/register	HTTP 200	HTTP 200 PASSED
POST /auth/register with valid data	HTTP 200 after redirect (auto-login)	HTTP 200 PASSED
POST /auth/register password mismatch	HTTP 200, 'do not match'	HTTP 200, 'match' PASSED
POST /auth/login valid credentials	HTTP 302 redirect	HTTP 302 PASSED
POST /auth/login wrong password	HTTP 200, 'invalid' or 'error'	HTTP 200, 'invalid' PASSED
GET /auth/logout (authenticated)	HTTP 302 redirect	HTTP 302 PASSED
GET /auth/login (already authenticated)	HTTP 302 redirect	HTTP 302 PASSED
GET /auth/register (already authenticated)	HTTP 302 redirect	HTTP 302 PASSED

4.2 Dashboard & Public Routes (9 tests)

File: tests/controllers/test_dashboard_controller.py

Test Input	Expected Output	Actual Output
GET /dashboard/ (unauthenticated)	HTTP 302 redirect to login	HTTP 302, 'login' in Location PASSED
GET /dashboard/ (authenticated)	HTTP 200	HTTP 200 PASSED
POST /dashboard/favorites/add (unauth)	HTTP 302 or 401	HTTP 302 PASSED
POST favorites/add missing fields	HTTP 400	HTTP 400 PASSED
POST favorites/add then remove	Both HTTP 200, success=True	Both 200, success=True PASSED
POST favorites/remove nonexistent	HTTP 404	HTTP 404 PASSED
GET /dashboard/analyses (unauth)	HTTP 302 redirect	HTTP 302 PASSED
GET / (public career index)	HTTP 200	HTTP 200 PASSED
GET /resume/upload (unauth)	HTTP 302 redirect	HTTP 302 PASSED

5. Black Box — Functional Testing: Models

Source files: tests/models/test_user.py, tests/models/test_career.py

5.1 User Model (8 tests)

Test Input	Expected Output	Actual Output
Create user with email, name, password	user.id not None, fields match	id set, fields match PASSED
Set password, check hash	hash != plaintext, check_password True/False	As expected PASSED
Duplicate email insert	Exception raised	Exception raised PASSED
update_last_login()	last_login not None	last_login set PASSED
repr(user)	Contains email string	Contains email PASSED
has_resume() on new user	Returns False	False PASSED
update_resume_path('/tmp/r.pdf')	Path set, timestamp set, has_resume=True	All set PASSED
delete_resume()	Path=None, timestamp=None, has_resume=False	All cleared PASSED

5.2 Favorite Courses via Model (6 tests)

Test Input	Expected Output	Actual Output
Add favourite course	result=True, len(favs)=1	True, len=1 PASSED
Add duplicate favourite	result=False, len(favs)=1	False, len=1 PASSED
Remove existing favourite	result=True, len(favs)=0	True, len=0 PASSED
Remove nonexistent favourite	result=False	False PASSED
Get favourites (2 added)	len=2	len=2 PASSED
Get favourite course IDs	Set {'c001', 'c002'}	{'c001', 'c002'} PASSED

5.3 User Skills (3 tests)

Test Input	Expected Output	Actual Output
set_skills([Python, SQL])	len(stored)=2	len=2 PASSED
set_skills replaces existing	Old skills gone, new ones present	Python gone, Java+Go present PASSED
get_skill_names()	Returns ['React']	['React'] PASSED

5.4 Resume Analysis Model (5 tests)

Test Input	Expected Output	Actual Output
Create analysis with score=7	id not None, score=7	id set, score=7 PASSED
get_grade_breakdown valid JSON	Returns parsed dict	{total: 85} PASSED
get_grade_breakdown invalid JSON	Returns None	None PASSED
get_grade_breakdown None field	Returns None	None PASSED
get_resume_analyses (3 added)	len=3	len=3 PASSED
get_latest_analysis (3 added)	Returns last inserted	latest.id = last id PASSED

5.5 Career Models — Industry, Course, DashboardStats (10 tests)

Test Input	Expected Output	Actual Output
Industry(growth=+12.5).get_growth_display()	'+12.5%'	'+12.5%' PASSED
Industry(growth=-3.2).get_growth_display()	'-3.2%'	'-3.2%' PASSED
Industry(growth=0.0).get_growth_display()	'+0.0%'	'+0.0%' PASSED
Course() — auto-generated course_id	course_id not None, len=12	len=12 PASSED
Same title+provider = same course_id	c1.course_id == c2.course_id	Equal PASSED
Different title+provider = different id	c1.course_id != c2.course_id	Not equal PASSED
Explicit course_id preserved	course_id='my_custom_id'	'my_custom_id' PASSED
get_subsidy_percentage(1000,200)	80	80 PASSED
get_subsidy_percentage free / zero	100 / 0	100 / 0 PASSED
DashboardStats creation	All fields stored	All match PASSED

6. Black Box — Functional Testing: Services

These are the service-layer tests covering core business logic.

6.1 AuthenticationService (15 tests)

File: tests/services/test_auth_service.py

Test Input	Expected Output	Actual Output
register_user valid inputs	success=True, user.email='new@example.com'	True, email matches PASSED
register_user email no @	success=False, 'email' in msg	False PASSED
register_user empty email	success=False	False PASSED
register_user short password (5)	success=False, '6 characters'	False, '6 characters' PASSED
register_user empty password	success=False	False PASSED
register_user short name (1 char)	success=False, 'name' in msg	False PASSED
register_user empty name	success=False	False PASSED
register_user duplicate email	success=False, 'already registered'	False, 'already registered' PASSED
register_user uppercase email	Stored as lowercase	user.email lowercase PASSED
register_user whitespace email	Trimmed	email trimmed PASSED
authenticate_user valid	success=True, user returned, last_login set	True, last_login set PASSED
authenticate_user wrong password	success=False, 'invalid'	False, 'invalid' PASSED
authenticate_user nonexistent email	success=False	False PASSED
authenticate_user empty credentials	success=False	False PASSED
authenticate_user case-insensitive	success=True	True PASSED

6.2 CourseService (34 tests)

File: tests/services/test_course_service.py

6.2.1 _extract_skills_from_title (5 tests)

Test Input	Expected Output	Actual Output
title='Advanced Python Programming'	'Python' in skills	Python in skills PASSED
title='Python for Data Analytics'	Multiple skills returned	Python + Data/Analytics PASSED
title='Intro to Basket Weaving'	['Professional Development']	['Professional Development'] PASSED
title with 9+ keywords	len(skills) ≤ 4	len ≤ 4 PASSED
title='Data and data analytics'	No duplicate 'Data'	count('Data')=1 PASSED

6.2.2 *apply_filters* (12 tests)

Test Input	Expected Output	Actual Output
filters=None	len=3, unchanged	len=3 PASSED
filters={} (empty dict)	len=3, unchanged	len=3 PASSED
impact_level='High Impact'	len=2, all High	len=2, all High PASSED
mode='Full-time'	len=1, title='B'	len=1, 'B' PASSED
language='Mandarin'	len=1, title='B'	len=1, 'B' PASSED
impact='All Impact Levels'	len=3, not filtered	len=3 PASSED
mode='All Modes'	len=3, not filtered	len=3 PASSED
language='All Languages'	len=3, not filtered	len=3 PASSED
sort_by='Highest Rating'	First item rating=4.5	First=C (4.5) PASSED
sort_by='Lowest Price'	First item lowest sub price	First=B (60) PASSED
sort_by='Shortest Duration'	First item=10 hours	First=A (10h) PASSED
Combined: High Impact + Highest Rating	len=2, first=C	len=2, first=C PASSED

6.2.3 *Parse Helpers* (17 tests)

Test Input	Expected Output	Actual Output
_safe_int('42', 0)	42	42 PASSED
_safe_int('42.7', 0)	42	42 PASSED
_safe_int(None, 99)	99	99 PASSED
_safe_int('abc', 99)	99	99 PASSED
_safe_int('None', 99)	99	99 PASSED
_safe_str(' hello ', 'def')	'hello'	'hello' PASSED
_safe_str(None, 'default')	'default'	'default' PASSED
_safe_str('None', 'default')	'default'	'default' PASSED
_clean_text('a_x000D_b')	'_x000D_' removed	No _x000D_ PASSED
_clean_text(None)	None	None PASSED
_parse_duration('40')	'40 hours'	'40 hours' PASSED
_parse_duration('0')	'N/A'	'N/A' PASSED
_parse_duration(None)	'N/A'	'N/A' PASSED

Test Input	Expected Output	Actual Output
_parse_rating('4.5')	4.5	4.5 PASSED
_parse_rating('6.0') (clamped)	5.0	5.0 PASSED
_parse_rating('0.2') (clamped)	1.0	1.0 PASSED
_parse_rating(None)	Random 4.3-4.9	$4.3 \leq r \leq 4.9$ PASSED

6.2.3 Parse Helpers continued — `_parse_reviews`, `_parse_impact`

Test Input	Expected Output	Actual Output
<code>_parse_reviews('150')</code>	150	150 PASSED
<code>_parse_reviews(None)</code>	Random 100-2000	$100 \leq r \leq 2000$ PASSED
<code>_parse_impact('4.5', 1000)</code>	'High Impact'	'High Impact' PASSED
<code>_parse_impact('3.5', 1000)</code>	'Medium Impact'	'Medium Impact' PASSED
<code>_parse_impact('2.0', 1000)</code>	'Low Impact'	'Low Impact' PASSED
<code>_parse_impact('0', 15000)</code>	'High Impact' (expensive)	'High Impact' PASSED
<code>_parse_impact('0', 500)</code>	'Medium Impact' (cheap)	'Medium Impact' PASSED
<code>_parse_impact(None, 15000)</code>	'High Impact'	'High Impact' PASSED
<code>_parse_impact(None, 500)</code>	'Medium Impact'	'Medium Impact' PASSED

6.3 DataDownloader (9 tests)

File: tests/services/test_data_downloader.py

Test Input	Expected Output	Actual Output
Init with non-existent CSV dir	Directory created	os.path.isdir = True PASSED
Init with existing dir	No error	No exception PASSED
Init stores URLs	Attributes match	Attributes match PASSED
<code>_should_refresh</code> missing file	True	True PASSED
<code>_should_refresh</code> fresh file	False	False PASSED
<code>_should_refresh</code> stale file (2 days)	True	True PASSED
<code>_should_refresh</code> custom max_age	False at 24h, True at 1h	Correct PASSED
<code>download_job_vacancies</code> cached	success=True, path ends .csv	True, ends .csv PASSED
<code>download_courses</code> cached	success=True, path ends .xlsx	True, ends .xlsx PASSED

6.4 DataService (13 tests)

File: tests/services/test_data_service.py

Test Input	Expected Output	Actual Output
<code>_get_industry_keywords('information technology')</code>	Non-empty, contains 'python'	Contains python, software PASSED
<code>_get_industry_keywords('healthcare')</code>	Non-empty, contains medical/health	Contains relevant kw PASSED
<code>_get_industry_keywords('underwater basket weaving')</code>	Empty list	[] PASSED

Test Input	Expected Output	Actual Output
<code>_get_industry_keywords(None)</code>	Empty list	<code>[]</code> PASSED
<code>_extract_skills_from_title('Advanced Python...')</code>	'Python' in skills	Python in skills PASSED
<code>_extract_skills_from_title('Basket Weaving')</code>	['Professional Development']	['Professional Development'] PASSED
<code>_extract_skills_from_title (9+ keywords)</code>	<code>len ≤ 4</code>	<code>len ≤ 4</code> PASSED
<code>_apply_filters(courses, None)</code>	<code>len=2, unchanged</code>	<code>len=2</code> PASSED
<code>_apply_filters impact='High Impact'</code>	<code>len=2, all High</code>	<code>len=2, all High</code> PASSED
<code>_apply_filters mode='Part-time'</code>	<code>len=1, title='A'</code>	<code>len=1, 'A'</code> PASSED
<code>_apply_filters sort by rating</code>	First='High' (5.0)	First=High PASSED
<code>_apply_filters sort by price</code>	First='Cheap'	First=Cheap PASSED
<code>_apply_filters 'All Impact Levels'</code>	<code>len=2, not filtered</code>	<code>len=2</code> PASSED

6.5 FavoriteCourseService (15 tests)

File: tests/services/test_favorite_course_service.py

Test Input	Expected Output	Actual Output
<code>add_favorite success</code>	<code>result=True</code>	True PASSED
<code>add_favorite creates DB record</code>	Record found with correct fields	Record found PASSED
<code>add_favorite default language</code>	<code>course_language='English'</code>	'English' PASSED
<code>add_favorite custom language</code>	<code>course_language='Mandarin'</code>	'Mandarin' PASSED
<code>add_favorite duplicate returns False</code>	<code>result=False</code>	False PASSED
<code>add_favorite duplicate no second record</code>	<code>count=1</code>	<code>count=1</code> PASSED
<code>Two users favourite same course</code>	Both return True	Both True PASSED
<code>remove_favorite existing</code>	<code>result=True</code>	True PASSED
<code>remove_favorite deletes DB record</code>	Record is None	None PASSED
<code>remove_favorite nonexistent</code>	<code>result=False</code>	False PASSED
<code>remove only affects target user</code>	Other user's fav preserved	Other user still has fav PASSED
<code>get_favorites empty</code>	Empty list	<code>[]</code> PASSED
<code>get_favorites returns all</code>	<code>len=2</code>	<code>len=2</code> PASSED
<code>get_favorite_course_ids empty</code>	Empty set	<code>set()</code> PASSED
<code>get_favorite_course_ids</code>	<code>{'c001', 'c002'}</code>	<code>{'c001', 'c002'}</code> PASSED

6.6 IndustryKeywords (11 tests)

File: tests/services/test_industry_keywords.py

Test Input	Expected Output	Actual Output
INDUSTRY_KEYWORDS is non-empty	len > 0	len > 0 PASSED
All industries have 'keywords' list	All have non-empty list	All have list PASSED
Known industries present (12 checked)	All 12 in registry	All present PASSED
All keywords are non-empty strings	All str, len > 0	All valid PASSED
get_industry_keywords exact match	Returns keywords with 'python'	Contains python PASSED
get_industry_keywords partial substring	Non-empty result	Non-empty PASSED
get_industry_keywords word overlap	Non-empty result	Non-empty PASSED
get_industry_keywords unknown	Empty list	[] PASSED
get_industry_keywords(None)	Empty list	[] PASSED
get_industry_keywords("")	Empty list	[] PASSED
get_industry_keywords case-insensitive	Non-empty (HEALTHCARE)	Non-empty PASSED

6.7 IndustryService (9 tests)

File: tests/services/test_industry_service.py

Test Input	Expected Output	Actual Output
ICON_MAP is non-empty	len > 0	len > 0 PASSED
Known icons present (IT, healthcare, financial)	All 3 in map	All present PASSED
get_industries_data returns success	success=True, err=""	True, "" PASSED
Returns non-empty list	len > 0	len > 0 PASSED
Max 10 industries	len ≤ 10	len ≤ 10 PASSED
Sorted by vacancies desc	Each ≥ next	Correctly sorted PASSED
All have ≥ 1000 vacancies	All ≥ 1000	All ≥ 1000 PASSED
All have non-empty icon	len(icon) > 0	All have icons PASSED
Growth rate is float	isinstance(float)	All float PASSED

6.8 ResumeAnalysisService (8 tests)

File: tests/services/test_resume_analysis_service.py

Test Input	Expected Output	Actual Output
_mock_analysis with action verbs in text	success=True, score>0, verbs found	True, score>0, verbs found PASSED
_mock_analysis no action verbs	success=True, score=3, verbs=[]	True, score=3, [] PASSED
_mock_analysis many verbs	success=True, score≥8	True, score≥8 PASSED
is_allowed_file('resume.pdf')	True (also 'Resume.PDF')	True PASSED
is_allowed_file('resume.docx')	False (also .png, no ext)	False PASSED
_parse_analysis_response valid format	success=True, score=7, verbs found	True, score=7 PASSED
_parse_analysis_response 'None' verbs	success=True, verbs=[]	True, [] PASSED
_parse_analysis_response score clamped	score=10 for 15, score≥1 for 0	10 and ≥1 PASSED

6.9 ResumeGradingService (28 tests)

File: tests/services/test_resume_grading_service.py

Helper Functions (8 tests)

Test Input	Expected Output	Actual Output
_tokens('Hello World Python')	['hello', 'world', 'python']	['hello', 'world', 'python'] PASSED
_tokens('Built 3 APIs!')	'built' in, '3' not in	Correct PASSED
_sentences('First. Second! Third?')	len=3	len=3 PASSED
_sentences('')	[]	[] PASSED
_bullet_lines(text with 3 bullets)	len=3	len=3 PASSED
_bullet_lines('Just a plain paragraph.')	[]	[] PASSED
_has_quantity('revenue by 30%')	True	True PASSED
_has_quantity('Led the team')	False	False PASSED

Rule-Based Scorers (17 tests)

Test Input	Expected Output	Actual Output
score_action_oriented: many verbs (12+)	points=8, len(found)≥10	8, found≥10 PASSED
score_action_oriented: few verbs (2)	0 < points < 8	Correct range PASSED
score_action_oriented: no verbs	points=0, found=[]	0, [] PASSED
score_specifics: quantified bullets	points>0, quantified=3	Correct PASSED
score_specifics: no quantified bullets	quantified=0, points=0	0, 0 PASSED
score_specifics: empty text	points=0	0 PASSED

Test Input	Expected Output	Actual Output
score_over_usage: no repetition	points=6	6 PASSED
score_over_usage: heavy repetition	points<6	<6 PASSED
score_avoided_words: no filler	points=6	6 PASSED
score_avoided_words: excessive filler	points<6	<6 PASSED
score_page_count(1)	points=10	10 PASSED
score_page_count(2)	points=10	10 PASSED
score_page_count(3)	points=5	5 PASSED
score_page_count(4)	points=0	0 PASSED
score_contact_info: phone+email	points=10, both True	10, both True PASSED
score_contact_info: email only	points=5, email=True, phone=False	5, correct flags PASSED
score_contact_info: none	points=0	0 PASSED

Competency & AI Scorer Fallbacks (10 tests)

Test Input	Expected Output	Actual Output
score_competency('analytical') with kws	points>0, matched_keywords>0	Correct PASSED
score_competency no matching kws	points=0	0 PASSED
score_competency invalid category	points=0	0 PASSED
_fallback_positions with leadership kws	points>0, ai_graded=False	Correct PASSED
_fallback_positions no kws	points=0	0 PASSED
_fallback_extracurricular with kws	points>0	>0 PASSED
_fallback_extracurricular no kws	points=0	0 PASSED
_parse_json valid JSON	Parsed dict	Correct dict PASSED
_parse_json with markdown fences	Parsed despite fences	Correct PASSED
_parse_json invalid/None/empty	None	None PASSED

Full Grading Pipeline (3 tests)

Test Input	Expected Output	Actual Output
grade() with full resume text	All keys present, sections sum to total, impact/40, presentation/30, competencies/30	All keys, correct sums PASSED
grade() scores within bounds	0≤impact≤40, 0≤presentation≤30, 0≤competencies≤30, total=sum	All within bounds PASSED
grade("") empty text	total ≥ 0, no crash	total ≥ 0 PASSED

6.10 SkillExtractionService (12 tests)

File: tests/services/test_skill_extraction_service.py

Test Input	Expected Output	Actual Output
_extract_with_fallback: 'Python developer'	'Python' in names	Python in names PASSED
_extract_with_fallback: multiple skills	Python, JS, React, Docker, AWS all in	All found PASSED
_extract_with_fallback: case-insensitive	'PYTHON' detected as 'Python'	Python found PASSED
_extract_with_fallback: no skills	len=0	len=0 PASSED
_extract_with_fallback: categories assigned	Each skill has category	All have category PASSED
_extract_with_fallback: no duplicates	Python appears once only	count=1 PASSED
_extract_with_fallback: Java vs JavaScript	JavaScript found, Java not false-positive	Correct PASSED
_extract_with_fallback: soft skills	Leadership, Communication found	Both found PASSED
extract_skills fallback (Ollama offline)	source='fallback', skills found	source=fallback PASSED
SKILL_TAXONOMY all have categories	Every skill in SKILL_CATEGORIES	All present PASSED
All categories non-empty	len(cat) > 0	All non-empty PASSED
All regex patterns compile	No re.error	All compile PASSED